ITT 036 – Programming Assignment 1

Digital Signal Encoder and Decoder

Autumn 2026

Student: IT49 IT67 IT69 **Language:** Java (JDK 17+)
Enrollment: 2024BITE069 and 2024BITE067 **Visualization:** Java Swing and AWT (`javax.swing`, `java.awt`)

Objective

Develop a modular Java application that generates digital bitstreams, finds the longest palindromic substring in linear time, performs digital line encoding and decoding, applies AMI scrambling, and demonstrates analog-to-digital conversion using Pulse Code Modulation (PCM) and Delta Modulation (DM).

Implemented Features

- **Digital Data Generation & Palindrome Finder:** Generates random binary sequences, sequences seeded with four (`0000`) or eight (`00000000`) consecutive zeros, or custom user inputs. Finds the longest palindromic substring in $O(n)$ time using **Manacher's Algorithm**.
- **Line Coding & Decoding:** Implements **NRZ-L**, **NRZ-I**, **Manchester**, **Differential Manchester**, and **AMI**. Includes extra-credit decoders that reconstruct the digital stream by analyzing encoded signal voltage levels and transitions rather than reading raw input bits.
- **Scrambling & Descrambling:** Supports single-pass $O(n)$ **B8ZS** (`000VB0VB`) and **HDB3** (`000V` / `B00V`) scrambling for AMI signals, along with violation-detecting descramblers.
- **Analog-to-Digital Conversion (PCM & DM):** PCM quantizes analog samples into fixed-width binary codewords and reconstructs a midpoint staircase signal. Delta Modulation tracks analog input using a 1-bit step size (Δ) and reconstructs the staircase approximation.
- **Visualization & Metrics:** Plots digital waveforms and reconstructed analog signals using Java Swing, and computes Baud rate, DC component, and transition density.

Design and Assumptions

- **Modular Structure:** The project separates logic into six packages (`data`, `encoding`, `decoding`, `scrambling`, `modulation`, and `visualization`) using interfaces and lookup maps instead of long `if/else` or `switch` chains.
- **Signal Representation:** NRZ and AMI schemes use 1 sample per bit ($\{+1, 0, -1\}$), while Manchester and Differential Manchester use 2 half-bit samples per bit to represent mid-bit transitions.
- **Initial Polarity:** NRZ-I and Differential Manchester start with a $+V$ reference level; AMI assumes the previous mark was $-V$ so the first `1` bit is encoded as $+V$. Analog inputs are normalized to $[-1.0, 1.0]$.

How to Compile and Run

1. Open the project root folder in a terminal (or VS Code integrated terminal) with JDK 17+ installed.
2. Compile all source files into the `bin/` folder and run `Main`:

```
javac -d bin src/Main.java src/data/*.java src/encoding/*.java src/decoding/*.java \
            src/scrambling/*.java src/modulation/*.java src/visualization/*.java
java -cp bin Main
```

3. Follow the interactive prompts to choose digital or analog input, encoding scheme, scrambling, and decoding.
4. To run the automated test suite (20 tests), execute: `java -cp bin ProjectTest`

Testing and Verification

All modules were verified using `ProjectTest.java` and dedicated unit tests (`EncodingDecoderTest`, `B8ZSKnownCasesTest`, `HDB3KnownCasesTest`, `DeltaModulationTest`). All 20 automated checks pass with exact round-trip bit recovery across all line coding schemes, B8ZS/HDB3 scrambling cases, and PCM/DM reconstructions.

References and Acknowledgments

- ITT 036: Programming Assignment 1 Specification, Autumn 2026.
- Forouzan, B. A., *Data Communications and Networking*, 5th Edition, McGraw-Hill Education.
- Oracle Java SE Documentation: `https://docs.oracle.com/en/java/`

- **Acknowledgment:** AI tools were used to assist with code documentation and LaTeX report formatting.